

Technical Assessment

MenteCart Service Booking with Cart

Full Stack Flutter Developer

Tech Stack • Flutter (BLoC) · Node.js · Express · MongoDB

This document contains the full project scope, requirements, suggested architecture, API surface, evaluation rubric, and submission checklist.

Contents

What's inside this brief

1. Project Overview
2. Tech Stack & Tooling
3. Core Features (Detailed)
4. Functional Requirements & Business Rules
5. Backend Architecture
6. REST API Surface
7. Frontend Architecture (Flutter)
8. Bonus Points (Worth Effort)
9. Submission Checklist

1. Project Overview

Build **MenteCart**, a service-booking application where users browse a catalogue of services (for example: home cleaning, plumbing, tutoring, beauty appointments), add one or more services to a cart, choose a date and time, and complete the booking with secure online payment.

Unlike a typical e-commerce cart, each line item in MenteCart represents a **time-bound service** with availability rules, capacity limits, and a status lifecycle. Your job is to design and implement both the Flutter mobile client and the Node.js (Express) backend that powers it.

What we're looking for

- Production-leaning code not a prototype. Treat it like a small but real product.
- Clear separation of concerns between UI, state, domain, and data layers.
- Thoughtful handling of edge cases: empty carts, overbooking, payment failure, network loss.
- Honest README that explains what's done, what's stubbed, and how to run it.
- Decisions documented briefly we care more about *why* than about a perfectly polished UI.

2. Tech Stack & Tooling

Frontend	Flutter (latest stable). Dart 3.x.
State Mgmt	Bloc (BLoC pattern) — required.
Backend	Node.js with Express
Language	TypeScript strongly preferred on the backend.
Database	MongoDB (Mongoose)
Auth	JWT access tokens (refresh tokens optional).

3. Core Features (Detailed)

3.1 User Authentication

Email + password signup and login, returning a signed JWT.

- Passwords hashed with **bcrypt** (≥ 10 salt rounds). Never store plain text.
- JWT issued on login, signed with a server-side secret, expiry 1–24h.
- Protected routes require Authorization: Bearer <token> header.
- Validate input on signup (email format, password min length).
- Return clear error codes 401 for bad credentials, 409 for email taken.

3.2 Browse Services

A catalogue endpoint plus a Flutter list / grid view.

- Each service has: id, title, description, price, duration, category, image, capacity per slot.
- Support filtering by category and basic search by title.
- Pagination on the API never return the entire table.
- Service detail screen showing description, price, available time slots.

3.3 Add to Cart

A cart that holds multiple services with selected date / time slots.

- Cart can live server-side (recommended) or client-side with sync on checkout.
- Adding the same service twice for the same slot should be prevented or merged.
- Show running total, count, and an itemised summary before checkout.
- Allow quantity / slot change and item removal.

3.4 Booking System

Convert cart items into bookings with a defined status lifecycle.

- Statuses: **pending** → **confirmed** → **completed**, plus **cancelled** and **failed**.
- Bookings created in pending state, transitioned to confirmed after payment success.
- Unpaid (cash / pay-on-arrival) bookings go straight to confirmed if business rules allow.
- Users can view their bookings and cancel before a defined cutoff.

4. Functional Requirements & Business Rules

These are the rules the system must enforce. They're the part of the assessment that separates a "works on my machine" CRUD app from a real booking system.

Cart rules

- A cart belongs to exactly one authenticated user.
- A cart can contain multiple services across multiple categories.
- Adding a service requires picking a date and a time slot not just an item.
- Cart items expire / release their held slot if not checked out within N minutes (suggest 15).

Booking limits

- Each user has a cap on bookings per day (configurable, e.g. max 3).
- Each service has a per-slot capacity once full, no new bookings for that slot.
- Cancellations release capacity back into the slot pool.

Preventing overbooking

- Capacity check + decrement must happen **atomically**. Race conditions are the failure mode we're looking for here.
- Use a transaction (Postgres) or a conditional update / findOneAndUpdate with a filter on remaining capacity (Mongo).
- If the atomic decrement fails, return 409 Conflict with a clear message — do not silently create a booking.

Paid vs unpaid bookings

- **Paid:** booking confirmed only after the PayHere webhook reports success (only applicable if you implement the optional PayHere bonus).
- **Unpaid:** confirmed immediately, flagged paymentMethod: 'cash' or 'pay_on_arrival' for the provider to settle.
- Both flows must reach the same confirmed end-state the difference is the paymentStatus.

Status management

- Status transitions must be **guarded** you can't go from cancelled back to pending.
- Every status change should be timestamped and ideally written to a small audit log.
- Failed payments should put the booking into failed and release held capacity.

5. Backend Architecture

Recommended layering

- **Routes / Controllers** HTTP only. Parse, validate, delegate. No business logic.
- **Services** business rules, transactions, orchestration.
- **Repositories / Models** database access only.
- **DTOs / Validators** Zod, class-validator, or Joi schemas at the boundary.
- **Middleware** auth (JWT verify), error handler, request logger.

6. REST API Surface

Below is the minimum endpoint set. You're welcome to add more, but every endpoint listed here should exist, be documented, and be tested manually (Postman / curl / Swagger) at minimum.

Method	Path	Purpose
POST	/auth/signup	Create user, return JWT
POST	/auth/login	Authenticate, return JWT
GET	/auth/me	Current user (protected)
GET	/services	List services (paginated, filterable)
GET	/services/:id	Service detail + slots
GET	/cart	Current user's cart
POST	/cart/items	Add item to cart
PATCH	/cart/items/:itemId	Update item (slot / qty)
DELETE	/cart/items/:itemId	Remove item from cart
POST	/bookings/checkout	Convert cart → pending booking
GET	/bookings	List user's bookings
GET	/bookings/:id	Booking detail
POST	/bookings/:id/cancel	Cancel booking (rules apply)

Conventions

- JSON request and response bodies. Use camelCase fields.
- Standard HTTP codes 200, 201, 400, 401, 403, 404, 409, 500. Don't return 200 with an error in the body.
- Errors follow a consistent shape: { statusCode, message, errorCode? }.
- Pagination via ?page=1&limit=20, response includes total and hasMore.

7. Frontend Architecture (Flutter)

State management

- **Bloc** is required — it forces explicit events and states, which we read as signal of discipline.
- Avoid setState for anything beyond local widget toggles.

Networking

- Use dio or http. Wrap in a single API client with interceptors.
- Auth interceptor attaches the JWT, refreshes on 401 if you implement refresh tokens.
- Centralised error mapping turn API errors into typed failures, not raw exceptions in the UI.
- Environment config (--dart-define) for the API base URL.

What we will check

- No secret keys should be committed to the repository. Use `.env` and `.env.example` files, and share sensitive data via one-time secret tools such as [<https://one-time-secret.de/en>]

8. Bonus Points (Worth the Effort)

Feature	Why it earns points
PayHere Payment Integration	Sandbox PayHere checkout, raw-body webhook signature verification, idempotent processing, and failure handling that releases capacity.
Docker setup	docker-compose.yml bringing up backend + db + redis with command
Logging	Structured logs (pino / winston) with request ids.

9. Submission Checklist

- Public GitHub repo, or invite us as a collaborator on a private one.
- Two folders / repos: /backend and /mobile (or two repos linked from a parent README).
- **README.md** at the root including: project summary, tech stack, prerequisites, env vars, step by-step run instructions, test card numbers used, known limitations.
- No secret keys should be committed to the repository. Use `.env` and `.env.example` files, and share sensitive data via one-time secret tools such as [<https://one-time-secret.de/en>]
- Demo video (Loom / YouTube unlisted) optional, 3–5 minutes walking through the happy path.

Good luck. We're more interested in clear thinking and clean execution on the core flow than in checking every bonus box.